

DB notes

- No index

Seq scan.

Execution time: 6.8ms

Sequential scan to discard 98k rows → not optimal

```
EXPLAIN ANALYZE
SELECT id, name, category, city, venue, event_date, created_at
FROM events
WHERE city = 'Athens' AND category = 'concert'
ORDER BY event_date ASC;
```

QUERY PLAN
Sort (cost=2907.75..2911.86 rows=1643 width=74) (actual time=6.639..6.706 rows=1646 loops=1)
Sort Key: event_date
Sort Method: quicksort Memory: 214kB
-> Seq Scan on events (cost=0.00..2820.00 rows=1643 width=74) (actual time=0.017..6.192 rows=1646 loops=1)
Filter: ((city = 'Athens'::text) AND (category = 'concert'::text))
Rows Removed by Filter: 98354
Planning Time: 0.859 ms
Execution Time: 6.841 ms

- With index

```
CREATE INDEX idx_events_city_category_date ON events (city, category, event_date);

EXPLAIN ANALYZE
SELECT id, name, category, city, venue, event_date, created_at
FROM events
WHERE city = 'Athens' AND category = 'concert'
ORDER BY event_date ASC;
```

QUERY PLAN
Sort (cost=1555.41..1559.52 rows=1643 width=74) (actual time=1.879..2.017 rows=1646 loops=1)
Sort Key: event_date
Sort Method: quicksort Memory: 214kB
-> Bitmap Heap Scan on events (cost=53.26..1467.66 rows=1643 width=74) (actual time=0.395..1.490 rows=1646 loops=1)
Recheck Cond: ((city = 'Athens'::text) AND (category = 'concert'::text))
Heap Blocks: exact=951
-> Bitmap Index Scan on idx_events_city_category_date (cost=0.00..52.85 rows=1643 width=0) (actual time=0.310..0.311 rows=1643 loops=1)
Index Cond: ((city = 'Athens'::text) AND (category = 'concert'::text))
Planning Time: 0.099 ms
Execution Time: 2.217 ms

Execution time 3x faster. Bitmap Heap Scan jumps to matches instead of Seq scanning.

- Ignoring left-most-prefix rule

```
EXPLAIN ANALYZE
SELECT id, name, category, city, venue, event_date, created_at
FROM events
WHERE category = 'concert'
ORDER BY event_date ASC;
```

QUERY PLAN
Sort (cost=3988.55..4038.22 rows=19870 width=74) (actual time=10.965..12.832 rows=19976 loops=1)
Sort Key: event_date
Sort Method: quicksort Memory: 2787kB
-> Seq Scan on events (cost=0.00..2570.00 rows=19870 width=74) (actual time=0.013..6.944 rows=19976 loops=1)
Filter: (category = 'concert'::text)
Rows Removed by Filter: 80024
Planning Time: 0.078 ms
Execution Time: 13.556 ms

Execution time gets slower again. PostgreSQL prefers Seq scan again instead of $O(\log(n))$ algorithm. All concert rows are scattered throughout the index. The index is useless.